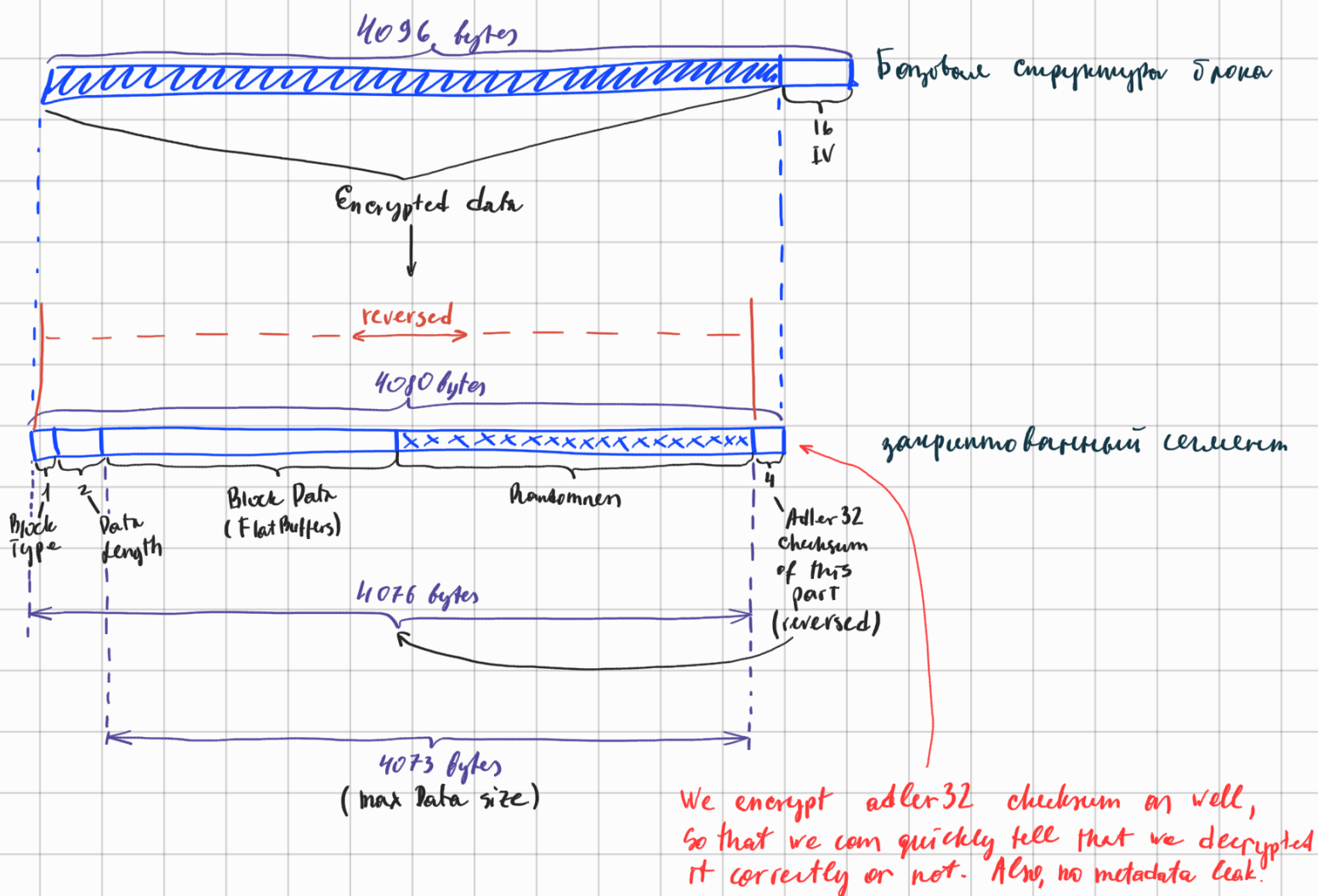




- ⑤ - **Key Block** - this block contains the encryption key to decrypt other blocks
- ③ - **Folders** - a tree-like structure of folders and subfolders
- ③ - **Symbol Sets** - sets of symbols for word generation (e.g. lower+upper+digits)
- ⑤ - **Phrase Templates** - templates for phrases (e.g. 'website' → 'url', 'login', 'password')
- **Phrases** - instances of templates (e.g. if 'website': 'gmail' → 'mail.ru', 'user', 'pass')
i.e. actual logins and passwords

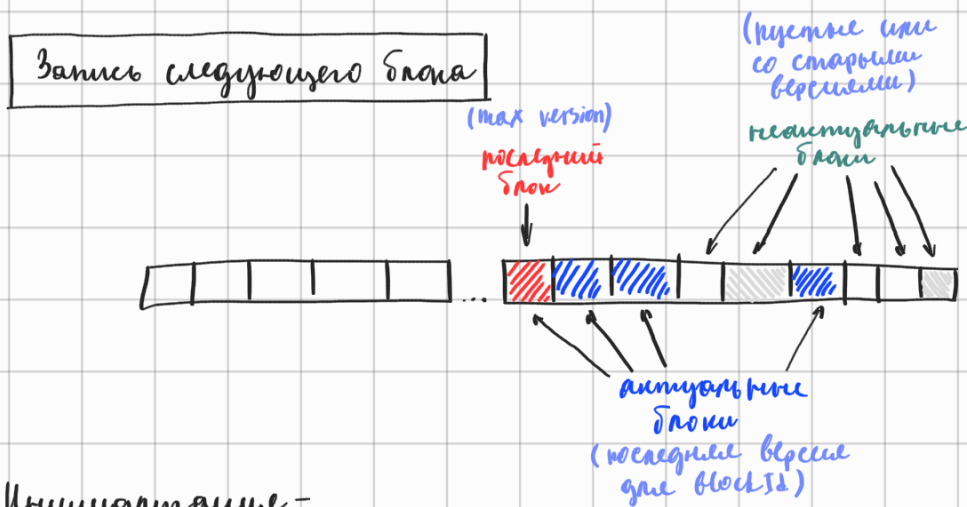
- We use AES-256 CBC with IV per block

- Revert Block byte array before encrypting - random bytes go first

complement with random bytes

reverse array

encrypt



Инициализация -

- Находим следующие блоки:

- 1) последний блок

- 2) следующий активный блок "справа" от блока (1)

- 3) следующий неактивный блок "справа" от блока (2)

- 4) следующий неактивный блок "справа" от блока (1), если не между блоком (1) и блоком (2) таких нет, берем сам блок (2)

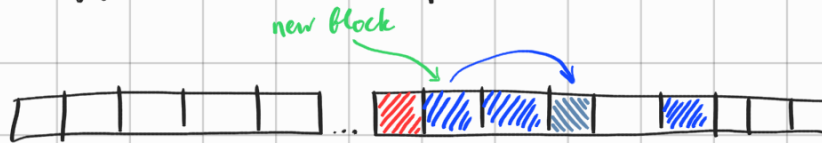
- Копируем блок (2) в блок (3), при этом обновим версию

- заметим, что при этом блок (2) становится неактивным, т.к. последняя версия теперь находится в блоке (3)

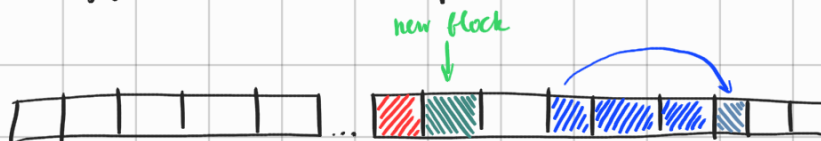
- записываем новый блок в блок (4) (который может совпадать с блоком (2))

- Напоследок:

- 1) Если следующий блок "справа" от блока (1) активен



- 2) Если следующий блок "справа" от блока (1) неактивен



- Это позволяет распределить нагрузку относительно равномерно по всей протяженности флеша, а кроме того предотвращает bit rot, характерный для флеш-памяти, путем копирования активных блоков.

- Можно также отдельно реализовать функцию обновления старейших активных блоков, для вызова пользователем из UI.

Улучшение секьюрности

① CleanUp/Defragment ✓ - OK

- Удалить все неактуальные блоки и оптимизировать структуру.

② Shuffle ✓ - OK

- Перемешать блоки для предотвращения атаки на KeyBlock, который без этой функции будет находиться в самом начале.

③ Triple AES \$ - дорого

- Тут придется писать свою имплементацию AES и это можно под микроконтроллер, это потребует времени, и существенно не изменит факт того, что оно и так секьюрно

table Folder
folder-id: uint16
parent-folder-id: uint16
folder-name: string

Folders

table Folders Block
block: StoreBlock
folders: [Folder]

table Key Block
block: StoreBlock
bucket-count: uint16
key: [byte]
db-name: [byte]
iv: [byte]

encryption key

table Symbol Set
set-id: uint16
symbol-set-name: string
symbol-set: string

Symbol Sets

table Symbol Sets Block
block: StoreBlock
symbol-sets: [SymbolSet]

struct StoreBlock
block-id: uint16
version: uint32
entropy: uint64

table Word Template
word-template-id: uint16
permissions: byte
icon: Icon
min-length: uint16
max-length: uint16
word-template-name: string
symbol-set-ids: [uint16]

Phrase Templates

table Phrase Template
phrase-template-id: uint16
phrase-template-name: string
word-template-ids: [uint16]

table Phrase Template Block
block: StoreBlock
word-templates: [WordTemplate]
phrase-templates: [PhraseTemplate]

Phrases

table Word
word-template-id: uint16
name: string
word: string
permissions: byte
icon: Icon

table Phrase History
phrase-template-id: uint16
phrase: [Word]

table Phrase Block
block: StoreBlock
phrase-template-id: uint16
folder-id: uint16
is-tombstone: bool
phrase-name: string
history: [PhraseHistory]

Permission flags

1 000x - TYPEABLE

2 00x0 - VIEWABLE

4 0x00 - GENERATEABLE

8 x000 - USER-EDITABLE

• enum BlockType; ← not used in structures

• enum Icon;